

\$whoami

Farhaan Bukhsh

Senior Software Developer @ OpenCraft
Open edX Release Manager



From Types to Trust: Safer API Design in Rust

Enums • Progressive Builders • Typestate • Capabilities

A case-study lens: Feluda (dependency & license analysis)



Running example: Feluda

A real CLI surface with many “valid combinations”

What Feluda does

- Analyzes project dependencies
- Collects dependency license metadata
- Flags restrictions / incompatibilities
- Multiple subcommands (scan, generate, sbom)
- Multiple ecosystems
(rust/node/go/python/c/cpp/r)



API constraints hiding in plain sight

- Some flags only apply to some commands
- Some commands require a project license
- “language” controls which manifests are valid
- output formats differ per feature (SBOM vs report)
- local-vs-network lookup changes behavior

Why APIs get misused

“Works in tests” ≠ “hard to misuse”

Common Anti-Patterns

- Stringly-typed inputs ("json" vs "JSON")
- Boolean flag soup (invalid combinations)
- Call-order rules hidden in docs
- Runtime validation scattered everywhere

Goal: make invalid states unrepresentable

Move constraints from runtime → types

If it compiles, it's probably valid.

If it's invalid, it shouldn't compile.

Pattern #1: Enums over strings

Make the set of valid values explicit

Stringly-typed

```
fn scan(language: &str) -> Result<()> {  
    match language {  
        "rust" => scan_cargo(),  
        "node" => scan_npm(),  
        _ => bail!("unknown language"),  
    }  
}
```

- Typos compile
- No autocomplete
- Harder refactors
- Error handling leaks everywhere

Enum-backed

```
enum Language { Rust, Node, Go, Python }  
  
fn scan(lang: Language) -> Result<()> {  
    match lang {  
        Language::Rust => scan_cargo(),  
        Language::Node => scan_npm(),  
        _ => todo!(),  
    }  
}
```

- Only valid values compile
- Autocomplete & docs via variants
- Refactors are compiler-guided
- Often removes Option/Result noise

Pattern #2: Progressive builders

Force required configuration before “run” exists

Builder stages encode “what’s missing” as types.



```
let report = Builder::new()  
  .language(Language::Rust)  
  .path("./proj")  
  .output(Output::SpdxJson)  
  .build()  
  .run()?;
```

Why it helps

- Eliminates “missing config” panics
- Guides users with autocomplete
- Makes required/optional fields obvious

Progressive Builder - ❌

```
impl ReportBuilder {
  fn new() -> Self {
    Self { path: None, format: None }
  }

  fn path(mut self, path: String) -> Self {
    self.path = Some(path);
    self
  }

  fn format(mut self, format: OutputFormat) -> Self {
    self.format = Some(format);
    self
  }

  fn build(self) -> Report {
    Report {
      path: self.path.expect("path required"),
      format: self.format.expect("format required"),
    }
  }
}
```

Progressive Builder -

```
impl NeedsPath {
  fn new() -> Self {
    Self
  }

  fn path(self, path: String) -> NeedsFormat {
    NeedsFormat { path }
  }
}

impl NeedsFormat {
  fn format(self, format: OutputFormat) -> ReadyToBuild {
    ReadyToBuild {
      path: self.path,
      format,
    }
  }
}

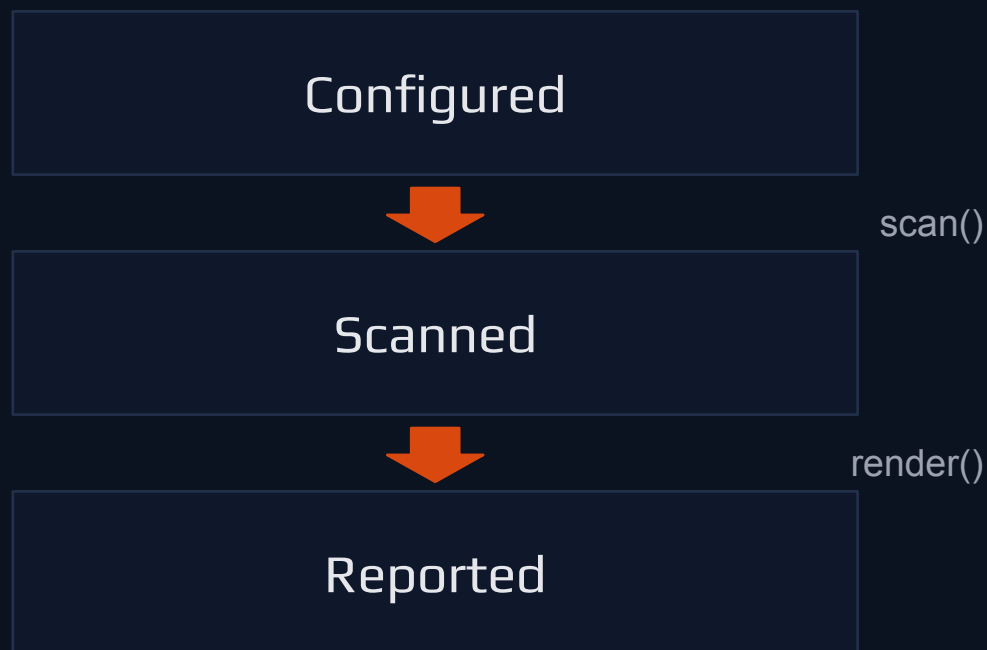
impl ReadyToBuild {
  fn build(self) -> Report {
    Report {
      path: self.path,
      format: self.format,
    }
  }
}

let report = NeedsPath::new()
    .path("licenses.json".into())
    .format(OutputFormat::Json)
    .build();
```

Pattern #3: Typestate

Different state = different type

Workflow encoded in types



Sketch

```
struct Configured { /* ... */ }
struct Scanned { /* ... */ }

impl Configured {
    fn scan(self) -> Scanned { /* ... */ }
}

impl Scanned {
    fn render(self) -> Report { /* ... */ }
}
```

- No “scan before configure”
- No “render before scan”
- State transitions are explicit

Typestate

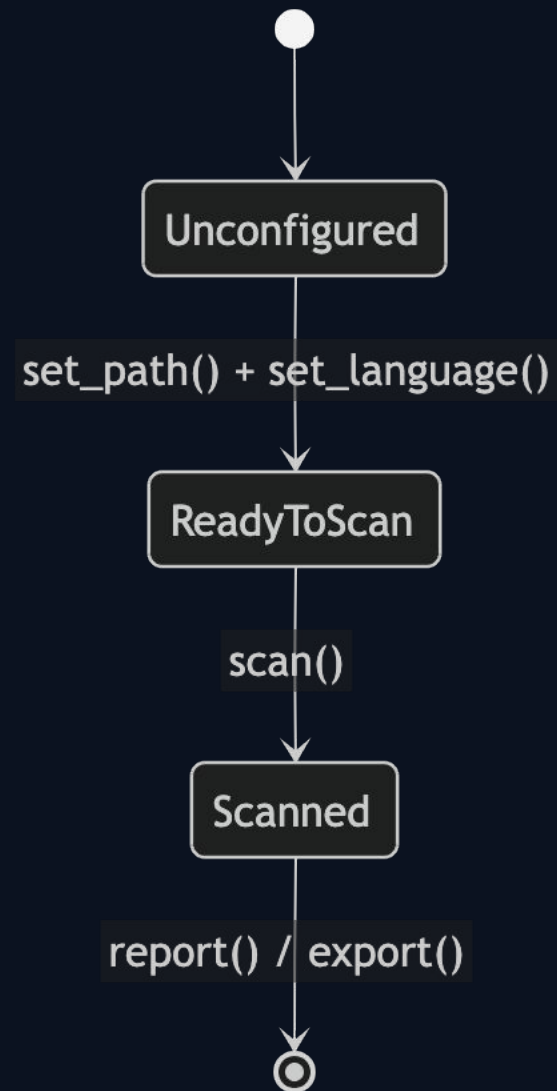
```
struct Unconfigured;
struct ReadyToScan;
struct Scanned;

struct Feluda<State> {
    state: State,
}

impl Feluda<Unconfigured> {
    fn new() -> Feluda<Unconfigured> {
        Feluda { state: Unconfigured }
    }
    fn configure(self) -> Feluda<ReadyToScan> {
        Feluda { state: ReadyToScan }
    }
}

impl Feluda<ReadyToScan> {
    fn scan(self) -> Feluda<Scanned> {
        Feluda { state: Scanned }
    }
}

impl Feluda<Scanned> {
    fn report(&self) {
        println!("Generating report");
    }
}
```



Pattern #4: Capability modeling

Permissions as types, not booleans

When different callers should have different powers:

ScanOnly

- read manifests
- fetch license data

PolicyCheck

- compat matrix
- restriction flags

Exporter

- write SBOM
- generate compliance files

Sketch

```
struct Feluda<C> { cap: C }

impl Feluda<ScanOnly> {
  fn scan(&self) -> Scanned { /*...*/ }
}

impl Feluda<Exporter> {
  fn write_sbom(&self) { /*...*/ }
}
```

Capability Modeling

```
trait CanScan {}
trait CanCheckPolicy {}
trait CanGenerateCompliance {}

struct InspectOnly;
struct PolicyMode;
struct ComplianceMode;

impl CanScan for InspectOnly {}
impl CanScan for PolicyMode {}
impl CanScan for ComplianceMode {}

impl CanCheckPolicy for PolicyMode {}
impl CanCheckPolicy for ComplianceMode {}

impl CanGenerateCompliance for ComplianceMode {}

impl<C> Feluda<C>
where
    C: CanCheckPolicy,
{
    fn check_policy(&self) {
        println!("Checking license restrictions and compatibility");
    }
}

let policy = Feluda {
    path: "./my-project".into(),
    language: Language::Rust,
    capability: PolicyMode,
};
```

Capability Modeling

```
struct Feluda<C> {
    path: String,
    language: Language,
    capability: C,
}

impl<C: CanScan> Feluda<C> {
    fn scan(&self) {
        println!("Scanning {:?} project at {}", self.language,
self.path);
    }
}

impl<C> Feluda<C>
where
    C: CanCheckPolicy,
{
    fn check_policy(&self) {
        println!("Checking license restrictions and compatibility");
    }
}

impl<C: CanGenerateCompliance> Feluda<C> {
    fn generate_compliance_file(&self) {
        println!("Generating compliance file");
    }
}
```

Putting it together

A cleaner library surface behind a CLI

Think in 3 layers

CLI parsing

strings, flags, env vars → typed config

Typed core API

enums • builders • typestate • capabilities

Execution

scan deps → analyze → render outputs

Payoff: The messy inputs stay at the edges; the core stays correct-by-construction.

Trade-offs & heuristics

Use types to remove common mistakes—not to show off

Use these patterns when...

- There are “valid combinations” people get wrong
- Order matters (configure → run → render)
- You maintain the API for a long time
- Misuse is costly
(security/compliance/reliability)

Avoid overengineering when...

- The workflow is tiny and obvious
- Types add more confusion than they remove
- Compile times / generics become painful
- The surface is only for internal one-off use

Pragmatic path: keep your CLI flexible, but make the core library typed.

Make the types pay rent

Docs, examples, and compile-fail tests

- Use rustdoc examples as the “happy path” tutorial
- Add doctests for the builder chain you want people to copy
- Use compile-fail tests for invalid states (trybuild/ui tests)
- Prefer small domain types (Path, LicenseId, OutputFormat) over &str

More Design/Types

For the curious ones...

- Session Types
- Phantom Data
- Newtypes
- Sealed traits
- Smart constructors / TryFrom

Wrap-up

A checklist for your next API

Ask these 5 questions

- What are the valid values? → enums
- What are the required inputs? → progressive builders
- What is the valid order of operations? → typestate
- Who should be allowed to do what? → capabilities
- What can we keep at the edges (parsing) vs core (meaning)?

Questions?

Connect / Follow

Type-Driven API Design in Rust • Feluda case study



Twitter/X

@fhackdroid



LinkedIn

farhaanbukhsh



GitHub

farhaanbukhsh



Instagram

farhaanbukhsh



Website

farhaan.bukh.sh

See you at Rust India Conference • Let's build in the open